

Assignment 3 (10% of total marks)

Due date: Sunday, 24 May 2026 by 9:00 pm Singapore time.

Scope:

This assignment includes the tasks in implementation of hierarchical data structures as BSON documents, implementation of simple queries in a query language of MongoDB database systems, implementation of aggregations and cursors, and implementation of data manipulations on BSON documents.

Assessment criteria:

Marks will be awarded for:

- Correct,
- Comprehensive, and
- Appropriate

application of the materials covered in this subject.

Please read carefully information listed below.

This assignment contributes to 10% of the total assessment mark for the subject CSCI235.

A submission procedure is explained at the end of specification.

This assignment consists of 2 tasks and specification of each task starts from a new page.

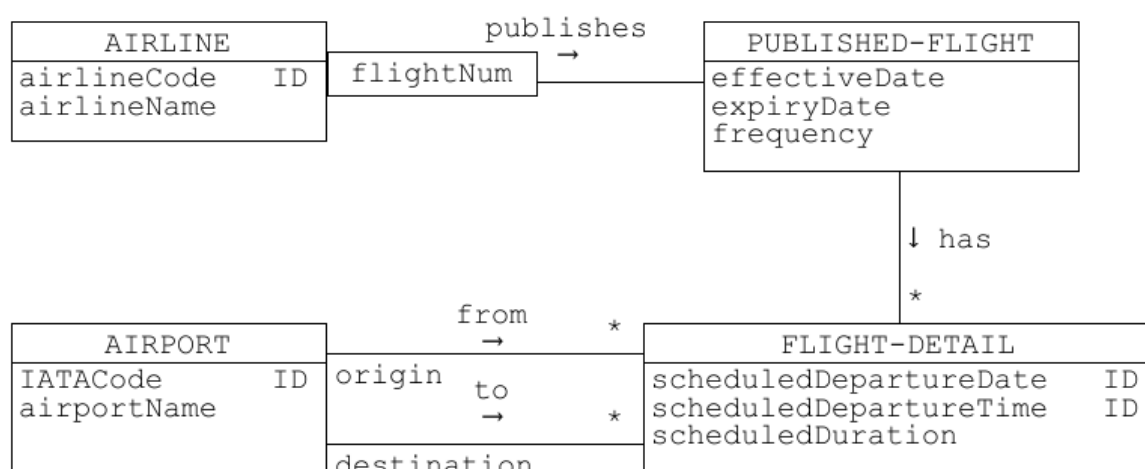
A policy regarding late submissions is included in the subject outline.

Assignment Specification:

Task 1 (3 marks)

Consider the conceptual schema of an airline system that records information about:

- AIRLINE
- PUBLISHED-FLIGHT
- FLIGHT-DETAIL
- AIRPORT



The system should store information about airlines, their published flights, the flight details for those flights, and the respective departing and arriving airports.

You are required to implement MongoDB documents that are consistent with the given conceptual schema.

Where possible, the documents should be modelled hierarchically. For example:

```

  AIRLINE
  └─ PUBLISHED-FLIGHT
      └─ FLIGHT-DETAIL
          └─ departingAirport
          └─ arrivingAirport
  
```

Class names and association names should also be represented clearly. For example, class names may be stored using a field such as `className`, while association names may be reflected using meaningful field names such as `publishedFlights`, `flightDetails`, `departingAirport`, and `arrivingAirport`.

The generated output file must be named: A3Sol1Output.txt

Your script must insert:

- At least 2 AIRLINE documents
- At least 3 embedded PUBLISHED-FLIGHT records in total
- At least 1 FLIGHT-DETAIL for each published flight
- Departing and arriving AIRPORT information for each flight detail

```
mongosh "mongodb://localhost:27017/myDB" --file
"D:/mongoDBScripts/A3Sol1.js" > D:/mongoDBScripts/A3Sol1Output.txt
```

The output file should display:

- Confirmation messages for insertions
- The number of documents inserted
- Pretty-printed documents retrieved from the collection

A file `A3Sol1Output.txt` with a report from processing of a script file `A3Sol1.js` that inserts the BSON documents into a collection and lists the documents in a pretty format. A report must have no errors and it must list all statements processed.

```
csci235> db.A3Task1.deleteMany({});
```

```
| db.A3Task1.insertMany([  
|   {  
|     "AIRLINE": {  
|       "airlineCode": "SQ",  
|       "airlineName": "Singapore Airlines",  
|       "Publishes": {  
|         "PUBLISHED-FLIGHT": [  
|           {
```

```

    "flightNum": "SQ222",
    "effectiveDate": "1 May 2026",
    "expiryDate": "31 May 2026",
    "frequency": 3,
    "Has": [
      {
        "FLIGHT-DETAIL": {
          "scheduledDepartDate": "28 May 2026",
          "scheduledDepartTime": "23:00",
          "scheduledDuration": 7,
          "ArrivalDepartAirport": {
            "arrivalAirport": {
              "IATACode": "SYD",
              "airportName": "Sydney Airport"
            },
            "departAirport": {
              "IATACode": "SIN",
              "airportName": "Singapore Changi Airport"
            }
          }
        }
      }
    ],
    {
      "flightNum": "SQ223",
      "effectiveDate": "1 May 2026",
      "expiryDate": "31 May 2026",
      "frequency": 3,
      "Has": [
        {
          "FLIGHT-DETAIL": {
            "scheduledDepartDate": "29 May 2026",
            "scheduledDepartTime": "10:00",
            "scheduledDuration": 7,
            "ArrivalDepartAirport": {
              "arrivalAirport": {
                "IATACode": "SIN",
                "airportName": "Singapore Changi Airport"
              },
              "departAirport": {
                "IATACode": "SYD",

```

```

    "airportName": "Sydney Airport"
  }
}
}
}
]
}
]
}
}
},
{
  "AIRLINE": {
    "airlineCode": "QF",
    "airlineName": "Qantas",
    "Publishes": {
      "PUBLISHED-FLIGHT": [
        {
          "flightNum": "QF082",
          "effectiveDate": "1 May 2026",
          "expiryDate": "31 May 2026",
          "frequency": 5,
          "Has": [
            {
              "FLIGHT-DETAIL": {
                "scheduledDepartDate": "30 May 2026",
                "scheduledDepartTime": "20:30",
                "scheduledDuration": 8,
                "ArrivalDepartAirport": {
                  "arrivalAirport": {
                    "IATACode": "SYD",
                    "airportName": "Sydney Airport"
                  },
                  "departAirport": {
                    "IATACode": "SIN",
                    "airportName": "Singapore Changi Airport"
                  }
                }
              }
            }
          ]
        }
      ]
    }
  }
}

```

```

|     ]
|     }
|     }
|     }
| ]);
|
| db.A3Task1.find().pretty();
[
{
  _id: ObjectId('6a186fa57fa173e0003682d1'),
  AIRLINE: {
    airlineCode: 'SQ',
    airlineName: 'Singapore Airlines',
    Publishes: {
      'PUBLISHED-FLIGHT': [
        {
          flightNum: 'SQ222',
          effectiveDate: '1 May 2026',
          expiryDate: '31 May 2026',
          frequency: 3,
          Has: [
            {
              'FLIGHT-DETAIL': {
                scheduledDepartDate: '28 May 2026',
                scheduledDepartTime: '23:00',
                scheduledDuration: 7,
                ArrivalDepartAirport: {
                  arrivalAirport: { IATACode: 'SYD', airportName: 'Sydney
Airport' },
                  departAirport: { IATACode: 'SIN', airportName: 'Singapore
Changi Airport' }
                }
              }
            }
          ]
        },
        {
          flightNum: 'SQ223',
          effectiveDate: '1 May 2026',
          expiryDate: '31 May 2026',
          frequency: 3,
          Has: [

```

```

    {
      'FLIGHT-DETAIL': {
        scheduledDepartDate: '29 May 2026',
        scheduledDepartTime: '10:00',
        scheduledDuration: 7,
        ArrivalDepartAirport: {
          arrivalAirport: { IATACode: 'SIN', airportName: 'Singapore
Changi Airport' },
          departAirport: { IATACode: 'SYD', airportName: 'Sydney
Airport' }
        }
      }
    }
  ]
}
]
}
},
{
  _id: ObjectId('6a186fa57fa173e0003682d2'),
  AIRLINE: {
    airlineCode: 'QF',
    airlineName: 'Qantas',
    Publishes: {
      'PUBLISHED-FLIGHT': [
        {
          flightNum: 'QF082',
          effectiveDate: '1 May 2026',
          expiryDate: '31 May 2026',
          frequency: 5,
          Has: [
            {
              'FLIGHT-DETAIL': {
                scheduledDepartDate: '30 May 2026',
                scheduledDepartTime: '20:30',
                scheduledDuration: 8,
                ArrivalDepartAirport: {
                  arrivalAirport: { IATACode: 'SYD', airportName: 'Sydney
Airport' },
                  departAirport: { IATACode: 'SIN', airportName: 'Singapore
Changi Airport' }
                }
              }
            }
          ]
        }
      ]
    }
  }
}

```

}
}
}
]
}
]
}
}
}
]
csci235>

Task 2 (7.0 marks)

Applying find or aggregation to search MongoDB documents.

Preliminary actions

The Assignment 3 folder contains a java scripts (subject.js) that creates a sample MongoDB database for task 2 of Assignment 3. Connect to MongoDB and execute the script subject.js. The database contains information about subjects their assessment and textbook. It is strongly recommended that you discover the conceptual schema (UML diagram) of the database. However, there will be no mark awarded for producing the conceptual schema.

Each question worth 0.6 marks. Make yourself familiar with the contents of the collections.

- 1) Find the subject with code "CSCI235".
- 2) Find all *Core* subjects with more than 3 assessments.
- 3) List the subject title, type, and total credit points for subjects worth at least 3 credits.
- 4) Find all subjects where "CSCI103" is listed as a prerequisite.
- 5) List the titles of all subjects whose assessment weight adds up to more than 80%.
- 6) Find all subjects that have at least one assessment with assessType equal to "Presentation".
- 7) For each subject, display: subCode, subtitle, totalAssessments.
- 8) For each subject, calculate the total weight of all assessments. The expected idea: CSCI103 total weight = 100.
- 9) Display all books where bookType is "textbook".
- 10) Find all subjects that have at least one book published by "Pearson"..

When ready, open a Terminal window and process your script at command shell prompt in the following way.

```
mongo -port 4000 databaseName < solution1.js > solution1.lst
```

Note, that there is no need to connect to MongoDB command line interface.

Sample solution includes but is not limited to the following:

```
print('Solution for A3, Task 2');
print('Student number: <UOW student number>');
print('Name: <Your Name>');
print('-----');

print('(1) Display all subject codes and titles from the collection.');
```

```
print("");
print('db.Subject.aggregate([{ $project: { _id: 0, Code: "$subject.subCode", Title:
"$subject.subTitle" } }]).toArray()');
print("");
printjson(
  db.Subject.aggregate([
    { $project: { _id: 0, Code: "$subject.subCode", Title: "$subject.subTitle" } }
  ]).toArray()
);
print("");
print('-----');
```

```
print('(2) Find all core subjects with more than 3 assessments.');
```

```
print("");
print('db.Subject.aggregate([{ $match: { "subject.type": "Core" } }, { $addFields:
{ numAssess: { $size: "$subject.assessments" } } }, { $match: { numAssess: {
$gt: 3 } } }, { $project: { _id: 0, subCode: "$subject.subCode", numAssess:1 } }
]).toArray()');
print("");
printjson(
  db.Subject.aggregate([
    { $match: { "subject.type": "Core" } },
    { $addFields: { numAssess: { $size: "$subject.assessments" } } },
    { $match: { numAssess: { $gt: 3 } } },
```

```
{ $project: { _id: 0, subCode: "$subject.subCode", numAssess: 1 } }
]).toArray()
);
print("");
print('-----');
print('(3) List the subject title, type, and total credit points for subjects worth at
least 3 credits.');
```

```
print("");
print('db.Subject.aggregate([ { $match: { "subject.type": "Core" } }, { $addFields:
{ numAssess: { $size: "$subject.assessments" } } }, { $match: { numAssess: {
$gt: 3 } } }, { $project: { _id: 0, subCode: "$subject.subCode", numAssess: 1 } }
]).toArray()');
```

```
print("");
printjson(
  db.Subject.aggregate([
    { $match: { "subject.credit": { $gte: 3 } } },
    { $project: { _id: 0, Title: "$subject.subTitle", Type: "$subject.type", Credit:
"$subject.credit" } }
  ]).toArray()
);
print("");
print('-----');
```

```
print('(4) Display the subject code and total number of assessments for each
subject.');
```

```
print("");
print('db.Subject.aggregate([ { $project: { _id: 0, subCode: "$subject.subCode",
numAssessments: { $size: "$subject.assessments" } } }]).toArray()');
```

```
print("");
printjson(
  db.Subject.aggregate([
    { $project: {
      _id: 0,
```

```
        subCode: "$subject.subCode",
        numAssessments: { $size: "$subject.assessments" }
    } }
]).toArray()
);
print("");
print('-----');
print('(5) List the titles of all subjects whose assessment weight adds up to more
than 80%.');
print("");
print('db.Subject.aggregate([{ $project: { _id: 0, subCode: "$subject.subCode",
numAssessments: { $size: "$subject.assessments" } } }]).toArray()');
print("");
printjson(
db.Subject.aggregate([
    { $addFields: {
        totalWeight: { $sum: "$subject.assessments.weight" }
    } },
    { $match: { totalWeight: { $gt: 80 } } },
    { $project: { _id: 0, Title: "$subject.subTitle", totalWeight: 1 } }
]).toArray()
);
print("");
print('-----');
print('(6) Find the subjects that include an Examination as part of their
assessment. Show subject code and title only');
print("");
print('db.Subject.aggregate([{ $match: { "subject.assessments.assessType":
"Examination" } }, { $project: { _id: 0, Code: "$subject.subCode", Title:
"$subject.subTitle" } }]).toArray()');
print("");
```

```
printjson(
db.Subject.aggregate([
  { $match: { "subject.assessments.assessType": "Examination" } },
  { $project: { _id: 0, Code: "$subject.subCode", Title: "$subject.subTitle" } }
]).toArray()
);
print("");
print('-----');
print('(7) List each subject together with its number of Assignments (assessType =
"Assignment").');
print("");
print('db.Subject.aggregate([ { $match: { "subject.assessments.assessType":
"Examination" } }, { $project: { _id: 0, Code: "$subject.subCode", Title:
"$subject.subTitle" } } ]).toArray());');
print("");
printjson(
db.Subject.aggregate([
  { $match: { "subject.assessments.assessType": "Examination" } },
  { $project: { _id: 0, Code: "$subject.subCode", Title: "$subject.subTitle" } }
]).toArray()
);
print("");
print('-----');
print('(8) Find all subjects that have Pearson as one of their book publishers.');
```

```
print("");
print('db.Subject.aggregate([ { $match: { "subject.book.publisher": "Pearson" } },
{ $project: { _id: 0, subCode: "$subject.subCode", Title: "$subject.subTitle" }
}]).toArray());');
print("");
printjson(
db.Subject.aggregate([
```

```
{ $match: { "subject.book.publisher": "Pearson" } },
{ $project: { _id: 0, subCode: "$subject.subCode", Title: "$subject.subTitle" } }
]).toArray()
);
print("");
print('-----');
print('(9) List the bookTitle, author(s), and publisher for all reference books
(bookType = "reference").');
print("");
print('db.Subject.aggregate([ { $unwind: "$subject.book" }, { $match: {
"subject.book.bookType": "reference" } }, { $project: { _id: 0, bookTitle:
"$subject.book.bookTitle", authors: "$subject.book.author", publisher:
"$subject.book.publisher" } } ]).toArray()');
print("");
printjson(
db.Subject.aggregate([
{ $unwind: "$subject.book" },
{ $match: { "subject.book.bookType": "reference" } },
{ $project: {
_id: 0,
bookTitle: "$subject.book.bookTitle",
authors: "$subject.book.author",
publisher: "$subject.book.publisher"
} }
]).toArray()
);
print("");
print('-----');
print('(10) Find subjects where at least one textbook was published before 2010.
Display the subject code and textbook title.');
```

```
print('db.Subject.aggregate([{ $unwind: "$subject.book" },{ $match:
{"subject.book.bookType": "textbook", "subject.book.yearPub": { $lt: 2010 } }},{
$project: { _id: 0,subCode: "$subject.subCode", textbook:
"$subject.book.bookTitle" } }]).toArray());

print("");

printjson(
db.Subject.aggregate([
  { $unwind: "$subject.book" },
  { $match: {
    "subject.book.bookType": "textbook",
    "subject.book.yearPub": { $lt: 2010 }
  } },
  { $project: {
    _id: 0,
    subCode: "$subject.subCode",
    textbook: "$subject.book.bookTitle"
  } }
]).toArray()
);

print("");

print('End of the implementation of Assignment 3.');
```

Submissions

This assignment is due by 9:00 pm (Singapore time) on Sunday, 24 May 2026.

Zip all your solutions (Solution1.js, Solution2.js, Solution1.pdf, Solution2.pdf) into a zipped file with a name <StudentNumber_A3.zip>. Submit the zipped file through Moodle in the following way:

- 1) Access Moodle at <http://moodle.uowplatform.edu.au/>
- 2) To login use a Login link located in the right upper corner the Web page or

- in the middle of the bottom of the Web page
- 3) When successfully logged in, select a site CSCI235 (SP226) Database Systems
 - 4) Scroll down to a section Submissions of Assignments
 - 5) Click at Submit your Assignment 3 here link.
 - 6) Click at a button Add Submission
 - 7) Move a file, for example, YourName_A3.zip into an area provided for your submission.
 - 8) Click at a button Save changes
 - 9) Click at a button Submit assignment
 - 10) Click at the checkbox with a text attached: By checking this box, I confirm that this submission is my own work, ... in order to confirm authorship of your submission.
 - 11) Click at a button Continue.

A policy regarding late submissions is included in the subject outline.

Only one submission per student is accepted.

Assignment 3 is an individual assignment and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification